Semantic-Lexical Fusion for Performance Bug Report Classification

Siddharth Shringarpure

Last updated: June 26, 2026

Abstract

Efficient bug report classification is increasingly important in software engineering as issue volumes grow beyond what manual triage can realistically handle. This creates bottlenecks in how issues are routed between maintainers, slowing down issue resolution. This report studies performance bug classification, where reports are either performance relevant (eg: accuracy, inference speed) or not. It evaluates a hybrid pipeline that combines sparse TF-IDF features with dense sentence embeddings against a TF-IDF-based Naïve Bayes baseline across five datasets, each corresponding to issues from a different machine-learning repository. A hybrid Logistic Regression model achieves higher macro-F1 across all datasets. Ablations show that most gains still hold when the sentence embedding dimension is reduced, whilst common preprocessing steps, such as stop-word removal and text lemmatisation, add little-to-no benefit.

1 Introduction

Identifying and routing bug reports is an essential software engineering task; as issue trackers accumulate growing numbers of free-text reports, automated classification reduces manual triage and allows developers to focus on fixing bugs, rather than organising reports. This report considers the binary classification of performance-related bug reports using five open-source machine-learning projects: `Caffe`, `Incubator-MXNet`, `Keras`, `PyTorch`, and `TensorFlow`.

A key challenge in text classification is document representation: traditional lexical representations — such as Bag of Words (BoW) and TF-IDF — are widely used for their simplicity and efficiency, but weight terms based on their distribution across the dataset rather than their semantic representation [1, 2, 3]. BoW representations do not handle synonymy or polysemy well [2], which is problematic for bug reports where similar issues might be expressed with different wording, whilst still containing useful project-specific cues. Lexical features are efficient to compute but limited in semantic discrimination, whilst dense vector embeddings capture meaning, but may underweight such cues. This motivates evaluating a hybrid sparse-dense representation, rather than treating lexical and semantic features as competing alternatives.

This report investigates whether a joint TF-IDF and sentence embedding classifier outperforms a TF-IDF-based Naïve Bayes (NB) baseline in distinguishing performance-related bug reports from others. Specifically, the evaluation centres on three key questions: are observed improvements consistent across projects, are these improvements maintained when the dimensionality of dense vector embeddings is reduced, and what trade-offs arise in terms of runtime, memory, and preprocessing? Guided by these questions, the report makes two contributions:

(1) a reproducible hybrid text-classification pipeline; and

(2) a paired, multi-project evaluation examining the source of observed performance gains.

The nature of performance bug reports motivates a hybrid representation. On one hand, they contain project-specific lexical signals, such as version numbers, configuration flags, and API identifiers, whose discriminative value lies in exact-token presence rather than meaning; these are precisely the cues that sparse TF-IDF captures and that dense encoders, trained for semantic similarity, tend to underweight. On the other hand, the nautral language *description* of a performance problem is lexically inconsistent across bug reporters — "slow", "takes forever", "relatively high latency" all denote the same underlying class — which is the case that semantic sentence embedding representation handles well and exact-match methods do not. Neither representation alone is sufficient: lexical models miss alternative phrasings, and semantic models underweight the project-specific tokens that often decide the label.

2 Related Work

Classical bug and issue classification has largely relied on sparse lexical features, such as TF-IDF, paired with conventional classifiers like NB and linear SVM [4, 5]. These approaches have been applied to related tasks, including GitHub issue-type prediction [6] (bug report vs feature request vs question), as well as distinguishing non-functional bug

reports from other issues using attention-based architectures [7]. Whilst lexical representations are efficient and capture project-specific terminology well, they are less effective at capturing semantic similarity between differently-worded reports. More recent work therefore shifts towards semantic representations. Sentence embeddings map reports with similar meanings closer together in representation space, making them useful for classification and retrieval [8], although they introduce additional computational cost and remain constrained by encoder input length [8, 9]. Techniques such as Matryoshka Representation Learning (MRL) take this further by training encoders to preserve information at multiple granularities within a single nested embedding, meaning that embeddings can be truncated with limited accuracy loss [10]. This points towards a middle ground approach, combining sparse lexical features — which capture project-specific tokens — with dense representations that better reflect semantic similarity.

3 Solution

The solution compares a simple lexical baseline with stronger linear and hybrid models. Bug reports are represented using both project-specific wording and broader semantic information, and the experiments test whether the classifier can leverage both.

3.1 Document Representation and Preprocessing

Issues are represented by concatenating the `title` and `body` into a combined text field, with the title placed first so it carries slightly more weight. Preprocessing is kept relatively minimal to avoid losing technical detail: text is lowercased, and noise like URLs and punctuation are removed. Non-ASCII characters are also filtered out, with extra whitespace normalised. Stop-words are not removed by default, as this could remove negation and modal cues (such as `not`, `no`, `should`), which are often useful for distinguishing performance issues; removing such terms can degrade classification performance.

3.2 Feature Fusion and Classification Strategy

As a baseline, a Gaussian NB classifier trained on unigram and bigram TF-IDF features is used, which serves as the lexical benchmark against which the hybrid model is evaluated. Logistic Regression (LogReg) is introduced as a stronger linear classifier, which was trained separately on TF-IDF features, sentence embeddings, and their combination. LogReg is chosen because it handles high-dimensional text features reliably [5] and is relatively easy to interpret. The main model is a `Hybrid LogReg` classifier that is trained on a concatenated representation combining sparse and dense features. The design targets improvements in classification performance, without adding significant architectural complexity.

Whilst TF-IDF captures project-specific terms like API names and error strings, sentence embeddings help group reports by meaning, even when the wording differs. A dual representation could therefore improve linear separability between performance and non-performance reports, compared with lexical features alone. Feature-level fusion is used over late fusion, as it allows a single classifier to jointly learn from both types of signals, rather than treating them as separate modalities with independent processing and output-stage aggregation.

3.3 Embedding Model and Dimensionality Design

Dense semantic representations of the concatenated report text were produced by the pre-trained sentence embedding model, `nomic-ai/nomic-embed-text-v1.5`. This model was chosen because prior work on the Nomic Embed family reports competitive results on both short- and long-context benchmarks [9].

Because of the encoder's bounded input length, reports were capped at 256 tokens before embedding. This reflects a practical computational constraint rather than an ideal representational choice: it keeps repeated inference manageable during benchmarking, at the cost of truncating later parts of longer reports. A further motivation for this model choice is that it was trained using Matryoshka Representation Learning, which enables lower-dimensional truncation with limited accuracy loss, thus retaining much of the original representation's utility. This makes it possible to study a quality-efficiency trade-off without additional model training or extra inference passes. Unless otherwise stated, the main experiments use the full 768-dimensional encoder output, with truncated versions examined separately in subsection 5.4.

4 Setup

The setup is designed to answer three questions: (1) whether the hybrid model improves on the baseline; (2) whether any gains are consistent across projects and random splits; and (3) what quality–efficiency trade-offs emerge under ablations of embedding dimension and preprocessing techniques. The first two address the main research questions directly, whilst the third assesses the practical significance of any observed gains. Classification performance is evaluated using paired

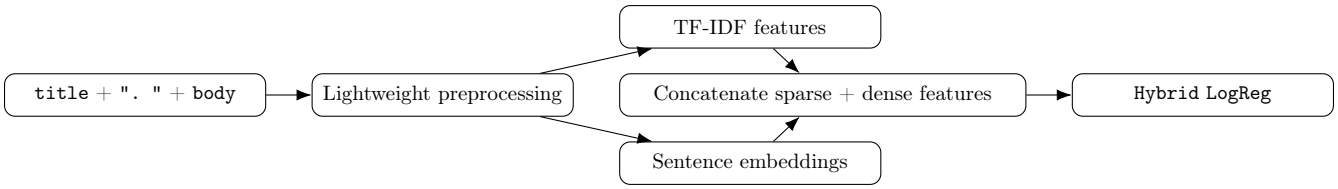

Figure 1: Hybrid pipeline: issue text is mapped to TF-IDF and sentence embedding features, then concatenated before classification.

runs and statistical testing to assess whether differences between models are statistically significant. Computational efficiency is measured via model-stage runtime (excluding one-off embedding generation) and peak Python memory usage. Additionally, embedding dimension ablations are performed to characterise the quality–efficiency trade-off, and a preprocessing ablation evaluates whether heavier preprocessing (ie: stop-word removal, text lemmatisation) improves on the main pipeline.

4.1 Methodology

Datasets from five open-source machine-learning projects are used to train the classifiers. Table 1 summarises the dataset sizes and positive-class proportions. The positive class (performance bug-related reports) is the minority class for every project, ranging from 11.54% to 20.21%. Macro-F1 is therefore used as the primary evaluation metric due to its handling of class imbalance, with accuracy, macro precision, macro recall, and AUC retained as secondary metrics.

Project	Reports	Positive reports	Positive rate
Caffe	286	33	11.54%
Incubator-MXNet	516	65	12.60%
Keras	668	135	20.21%
PyTorch	752	95	12.63%
TensorFlow	1490	279	18.72%

Table 1: Dataset sizes and positive-class proportions.

Each dataset was evaluated over 30 runs using 70/30 stratified train-test splits, providing sufficient paired observations for stable statistical comparison. All models were tested on identical splits to ensure fair comparison, allowing per-run macro-F1 values to be compared directly under matched conditions; reported values are means over the 30 stratified runs.

TF-IDF features were fitted on the training text in each split, and then applied to the test text. This approach avoids fitting the vectoriser on the full dataset before splitting, as that would allow test-set data to influence the feature space prior to test-set evaluation, likely leading to overoptimistic performance estimates.

Sentence embeddings were generated using the pre-trained encoder described in subsection 3.3, and aligned to the same splits. The encoder was not trained nor fine-tuned during experiments.

4.2 Statistical Testing and Efficiency Measurement

Performance is evaluated using a one-sided paired Wilcoxon signed-rank test on per-run macro-F1 scores ($\alpha = 0.05$), comparing `Hybrid LogReg` against the baseline. Because both models are evaluated on the same train-test splits, the analysis uses paired differences in scores. The test is one-tailed, as the goal is to check whether the hybrid model performs better, rather than simply whether the models differ. Wilcoxon's test is preferred over a paired Student's t-test because the latter's normality assumption on score differences is unlikely to hold here. Because several model variants are compared on the same splits, a Friedman test is also used as an overall non-parametric test over the baseline, `TF-IDF + LogReg`, `embedding LogReg`, and `Hybrid LogReg`. Each run is treated as a matched set, since all models are evaluated on the same data split, allowing comparisons based on within-run rankings. This choice follows Demšar, who recommends non-parametric tests like Wilcoxon and Friedman under these conditions [11].

Computational efficiency is measured at the model-stage, rather than over full end-to-end runtime. In practice, this means the focus is on costs once features are prepared. Runtime is broken down into feature preparation, model fitting, and inference per run; peak Python memory usage is also recorded using `tracemalloc`. These measurements exclude the one-off cost of generating sentence embeddings, since these were cached and reused across runs. As a result, the efficiency figures reflect repeated evaluation with cached features, rather than a deployment or cold-start scenario.

5 Experiments

5.1 Main Comparison Across Models

Figure 2 summarises the main four-model comparison at the default dimensionality. All models outperform the baseline on macro-F1, with `Hybrid LogReg` achieving the highest scores across all datasets. Among the non-hybrid models, `Embedding LogReg` is generally strongest, with `TF-IDF + LogReg` marginally exceeding it on `Caffe`. Two additional methods are included to evaluate the semantic signal captured by dense vector embeddings: `Centroid` and k-nearest neighbours (`kNN`) classifiers, which serve as reference points for the dense embedding space in isolation.

Figure 2: Mean macro-F1 by dataset, including Late Fusion, Centroid, and kNN baselines.

The hybrid model improves over the lexical baseline on all datasets, with the largest gain on `Incubator-MXNet` (+0.341 macro-F1) and the smallest on `Caffe` (+0.165). Across secondary metrics (accuracy, macro precision, macro recall, and AUC), the same pattern broadly holds: `Hybrid LogReg` is the strongest overall, achieving the highest accuracy on all five datasets, the highest macro precision on 4 out of 5, and the highest AUC on 4 out of 5. The main exception is macro recall, where `Embedding LogReg` performs best across all datasets.

5.2 Late Fusion as a Supplementary Comparison

A `Late Fusion` LogReg variant was also evaluated as a supplementary comparison. Rather than training on concatenated features, this variant averages the output probabilities of `TF-IDF + LogReg` and `Embedding LogReg`, with equal weight. On every dataset, it performs better than either component model alone, but it does not outperform feature-level fusion on any dataset.

Relative to `Hybrid LogReg`, the macro-F1 gap is 0.0116 on `Caffe`, 0.0018 on `Incubator-MXNet`, 0.0030 on `Keras`, 0.0085 on `PyTorch`, and 0.0040 on `TensorFlow`, meaning that `Late Fusion` remains within 0.01 macro-F1 of the hybrid on four of the five datasets, but still underperforms feature-level fusion across all datasets.

5.3 Statistical Evidence

Table 2 shows the paired Wilcoxon results for `Hybrid LogReg` against the baseline, with the largest one-sided p-value remaining very small. The direction of the paired differences is also almost uniform: `Caffe` has positive deltas in 29 of 30 runs, whilst the other datasets have positive deltas in all 30 runs.

Dataset	Mean ΔF1	Improvement	p	Positive deltas
Caffe	0.1653	26.5%	2.794×10^{-9}	29
Incubator-MXNet	0.3412	67.9%	9.313×10^{-10}	30
Keras	0.3256	61.4%	9.313×10^{-10}	30
PyTorch	0.2852	53.9%	9.313×10^{-10}	30
TensorFlow	0.2540	40.5%	9.313×10^{-10}	30

Table 2: Paired Wilcoxon results for `Hybrid LogReg` vs `Baseline NB`.

The supplementary Friedman tests point in the same direction: all datasets show a statistically significant difference across the four main models, with $p \leq 5.73 \times 10^{-12}$ in every case, and `Hybrid LogReg` has the best mean rank on every dataset.

5.4 Effect of Reducing Embedding Dimension

Figure 3 shows how `Hybrid LogReg` behaves as the dense-vector component is truncated. The full embedding performs best overall, but much of the performance is retained even at lower dimensions. Staying within 0.01 macro-F1 of the

full embedding requires 512 dimensions for `Caffe`, 128 for `Incubator-MXNet` and `Keras`, and 256 for `PyTorch` and `TensorFlow`. The relationship between embedding dimension and macro-F1 is not strictly monotonically decreasing; for instance, `Incubator-MXNet` scores slightly higher at 128 dimensions than at 256. Within 0.01 macro-F1 of the full embedding, 256- or 512-dimensional configurations are sufficient for most datasets.

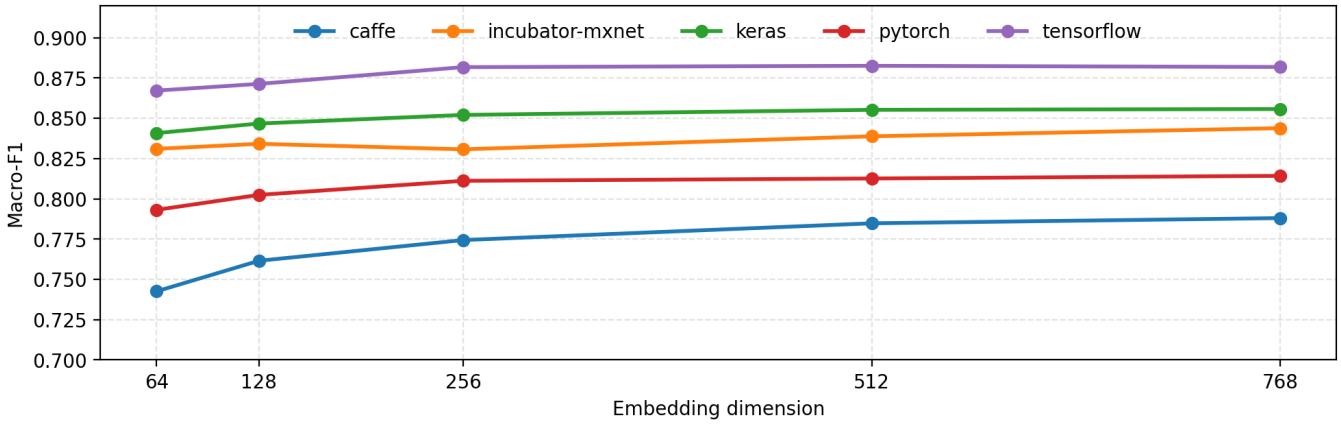

Figure 3: `Hybrid LogReg` macro-F1 across embedding dimensions.

5.5 Runtime and Memory Usage

Table 3 summarises average model-stage runtime and peak Python memory across the five datasets, excluding the one-off cost of generating embeddings. Under this cached benchmark, `Hybrid LogReg` achieves the highest mean macro-F1 among the main models. It is also the second-heaviest in runtime and peak Python memory usage, behind only the baseline. On the other hand, `Embedding LogReg` has the lowest repeated runtime and memory usage among the main learned models once embeddings have been cached. For `TF-IDF + LogReg` and both fusion models, repeated runtime is dominated by TF-IDF feature preparation; sentence embedding-based models avoid this cost once embeddings are cached.

Model	Mean macro-F1	Runtime (s)	Peak Memory (MB)
Baseline	0.5625	0.7683	28.07
TF-IDF + LogReg	0.7824	0.5008	11.22
Embedding LogReg	0.8184	0.0065	3.32
Late Fusion LogReg	0.8310	0.5073	11.22
Hybrid LogReg	0.8367	0.5266	13.39
Embedding Centroid	0.7886	0.0029	1.29
Embedding kNN	0.7948	0.0082	2.75

Table 3: Average cached model-stage efficiency across the datasets.

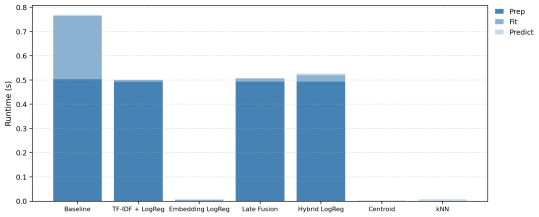

Figure 4: Average runtime profile by model.

The embedding-space baselines are still useful, despite how simple they are: `Embedding kNN` beats `TF-IDF + LogReg` on macro-F1 across all five datasets, and `Embedding Centroid` does so on four of them. They do not match `Hybrid LogReg`, but come at a noticeably lower runtime and memory usage cost.

5.6 Effects of Heavier Preprocessing

The preprocessing ablation applies the same base cleaning pipeline in all tested modes as described in Section 3.1, then applies mode-specific steps, as detailed in Table 4. The filtered stopword variant specifically does not contain some negation and modal terms. Text lemmatisation is the strongest non-default option, but it has negligible impact on the mean `Hybrid LogReg` macro-F1. The stop-word variants are slightly worse on average, with mean changes in the range $[-0.009, -0.010]$.

Mode	Mean ΔF1 vs default	Worst ΔF1
Default	0.0000	0.0000
Stopwords	-0.0092	-0.0155
Stopwords (filtered)	-0.0091	-0.0221
Lemmatise	0.0004	-0.0031
Stopwords (filtered) + Lemmatise	-0.0102	-0.0219

Table 4: Preprocessing variants and hybrid macro-F1 change vs default.

6 Discussion

6.1 Key Findings and Interpretations

Across all datasets, `Hybrid LogReg` is the strongest overall model, with consistent performance improvements over `Baseline NB`, showing that the improvement is not driven by a single dataset. The embedding-only models (centroid, kNN) also perform competitively, showing that dense vectors alone capture a large proportion of the task-relevant signal. However, the hybrid model achieves stronger performance, indicating that lexical features provide complementary information beyond semantics alone. The size of the performance gain varies across datasets, suggesting that the relative importance of sparse and dense features depends on dataset characteristics.

The truncation ablation shows that most of the performance is preserved even at lower embedding dimensions. The hybrid model does not rely heavily on the full 768-dimensional representation, with 256- or 512-dimensional variants performing similarly at lower cost. The relationship between dimensionality and performance is also not strictly monotonic, suggesting some redundancy in the embedding space. This highlights a trade-off: a hybrid approach delivers strong predictive performance, but at higher computational cost than simpler models, particularly dense-vector-only approaches.

Heavier normalisation does not improve performance; stop-word removal is slightly worse, and lemmatisation adds little beyond the existing representation. Feature-level fusion also outperforms late fusion across all datasets, suggesting that combining model features is more effective than averaging model outputs.

6.2 Limitations

This evaluation is limited to a small set of open-source machine-learning repositories. It focuses on a binary bug report classification task, so it does not cover cross-domain settings (ie: non-ML repositories), multi-label categorisation, or open-set issue triage. The positive class is restricted to performance-related bug reports, meaning that the results do not extend to broader bug-type taxonomies or finer-grained subtypes.

The model uses only the issue `title` and `body`, excluding other signals like comments, labels, and code snippets. The preprocessing pipeline is deliberately simple and removes punctuation and non-ASCII characters, which might discard useful formatting in technical text (eg: text-code separators, structured Markdown, formatted emphasis). Sentence embeddings are generated using a pretrained general-purpose encoder rather than one adapted to software engineering language; this keeps training cheaper and simpler, although a domain-specific encoder may perform better. The 256-token cap can also truncate longer issue bodies, potentially reducing representational fidelity.

The evaluation setup has further limits. Cost figures are based on repeated cached runs, rather than full deployment-time measurements, and the experiment focuses on sparse, dense, and fusion-based approaches, without extensive architectural variation. The datasets used also vary substantially in size (eg: `Caffe`: 286 reports, 33 positives), so results from smaller datasets should be interpreted more cautiously. Generally, the results support the hybrid design for the repositories used here, but additional evidence would be needed before generalising to other workflows or domains.

7 Future Work

The current design can be extended in several ways, for instance, expanding the input beyond the issue `title` and `body`. This could include labels, comments, or code snippets, and their addition could enable clearer attribution of performance gains to lexical, semantic, or structural cues. A further extension is to evaluate the pipeline under stricter generalisation conditions, using cross-project or temporally ordered splits. In the former, training data would be drawn from one set of repositories, and test data from a distinct, non-overlapping set; in the latter, training would use earlier reports and testing would use later ones. Both are stricter than the current random-split procedure and would better assess whether the observed gains persist under realistic deployment conditions.

Another direction is to revisit the semantic representation. Instead of relying on general-purpose sentence embeddings, the encoder could be fine-tuned for bug report classification using a lightweight supervised head or a contrastive objective. This may improve class separation, but at the cost of additional compute and a higher risk of overfitting, especially with limited labelled data. A final direction is to examine the 256-token cap alongside dense vector dimensionality. Whilst the ablation shows that most hybrid gains exist at lower dimensions, a more focused study could test whether errors are concentrated in longer reports affected by truncation, and whether reduced dimensionality remains acceptable under this constraint.

8 Conclusion

The main question explored whether combining TF-IDF features with sentence embeddings can materially outperform a NB + TF-IDF baseline on performance bug-related report classification. The results show that `Hybrid LogReg` achieves the highest macro-F1 across all datasets, with paired statistical tests supporting this outcome.

The dimension ablation shows that much of the hybrid gain is retained at embedding dimensions below the full 768, suggesting that improvements primarily stem from incorporating semantic information rather than embedding dimensionality itself. However, despite outperforming alternatives in predictive performance, the runtime and memory results show that this improved classification performance comes at a higher computational cost, which may affect deployment decisions where efficiency is essential.

Feature-level fusion also reliably outperforms late fusion, whilst embedding-based approaches remain competitive, confirming that semantic information is valuable, but most effective when combined with lexical features. Overall, the results support a clear conclusion: an architecturally simple sparse-dense classifier provides a consistent improvement over the baseline, whilst retaining most of its benefit under moderately reduced dense vector dimensions. Whilst the findings do not establish a universally optimal configuration across repositories or workflows, a simple hybrid pipeline offers developers consistent improvements, at the cost of increased computation.

9 Artefact

The repository, available at: `github.com/siddharth-shringarpure/buglex`, contains the full source code, datasets, and raw result CSV files used to generate the tables and figures in the report. The repository root also contains `requirements.pdf`, `manual.pdf`, and `replication.pdf`. Source code is under `src/`, datasets under `datasets/`, generated outputs under `results/`, and report materials under `docs/`.

References

[1] K. Sparck Jones, "A statistical interpretation of term specificity and its application in retrieval," *J. Doc.*, vol. 28, no. 1, pp. 11–21, 1972.

[2] M. A. Mouriño-García, R. Pérez-Rodríguez, L. Anido-Rifón, and M. Vilares-Ferro, "Wikipedia-based hybrid document representation for textual news classification," *Soft Comput.*, vol. 22, no. 18, pp. 6047–6065, Sep. 2018. [Online]. Available: http://link.springer.com/10.1007/s00500-018-3101-5

[3] J. Lilleberg, Y. Zhu, and Y. Zhang, "Support vector machines and Word2vec for text classification with semantic features," in *2015 IEEE 14th Int. Conf. Cognitive Informatics & Cognitive Computing (ICCI*CC)*. Beijing, China: IEEE, Jul. 2015, pp. 136–140. [Online]. Available: http://ieeexplore.ieee.org/document/7259377/

[4] A. McCallum and K. Nigam, "A comparison of event models for Naive Bayes text classification," in *AAAI Workshop on Learning for Text Categorization*, 1998.

[5] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin, "LIBLINEAR: A library for large linear classification," *J. Mach. Learn. Res.*, vol. 9, pp. 1871–1874, 2008.

[6] R. Kallis, A. Di Sorbo, G. Canfora, and S. Panichella, "Predicting issue types on GitHub," *Sci. Comput. Program.*, vol. 205, p. Art. no. 102598, May 2021. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0167642320302069

[7] G. Long, T. Chen, and G. Cosma, "Multifaceted hierarchical report identification for non-functional bugs in deep learning frameworks," 2022. [Online]. Available: https://arxiv.org/abs/2210.01855

[8] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conf. Empirical Methods Natural Language Process. and 9th Int. Joint Conf. Natural Language Process. (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, 2019, pp. 3980–3990. [Online]. Available: https://www.aclweb.org/anthology/D19-1410

[9] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, "Nomic Embed: Training a reproducible long context text embedder," Feb. 2025. [Online]. Available: http://arxiv.org/abs/2402.01613

[10] A. Kusupati, G. Bhatt, A. Rege, M. Wallingford, A. Sinha, V. Ramanujan, W. Howard-Snyder, K. Chen, S. Kakade, P. Jain *et al.*, "Matryoshka representation learning," *Adv. Neural Inf. Process. Syst.*, vol. 35, pp. 30 233–30 249, 2022.

[11] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, Dec. 2006.